

NYCU DL LAB3 Report

313551127 王翔

1. Overview:

In lab 3, we implemented two network architectures, UNet and ResNet34_UNet, for Binary Semantic Segmentation, which classifies each pixel as foreground or background. We used the Oxford-IIIT Pet Dataset and the default training, validation, and test sets. Additionally, we used the dice score as the metric to evaluate the prediction results. The models were based on the architectures described in the respective papers for these two models, with slight modifications to meet our needs, such as using 256 x 256 as the input image size. Both models achieved a dice score greater than 0.9, indicating that these models are very suitable for this task.

2. Implementation Details:

A. Details of your training, evaluating, inferencing code:

1.train.py:

At the beginning of the training, we used the `get_hyperparameters` function from the `utils` module to obtain all the hyperparameters needed for our training process. For this lab, I mainly used PyTorch's built-in `BCEWithLogitsLoss` and my custom-defined `DiceBCELoss` as the loss functions. The latter combines BCE loss and dice loss. Next, I set up the optimizer and scheduler, using Adam and Exponential LR, respectively. To track the training and validation loss and dice score, I utilized `summaryWriter`. I also employed gradient clipping techniques to prevent the gradient norm from becoming too large and learning.

After training all batches, I used the validation loader and an evaluating function to calculate the validation loss and dice score. Given the large model size, to save storage space and time, I

recorded the current best validation dice score using `best_dice`. When a better score was achieved, I updated and saved the best model. Additionally, to avoid overfitting, I employed early stopping when the model's dice score no longer improved.

```
def train(args):
    # implement the training function here
    #data_path epochs batch_size learning rate
    device = "cuda" if torch.cuda.is_available() else "cpu"
    stale = 0
    best_dice = 0
    hyperparameters = get_hyperparameters(args.model)
    if args.model == 'unet':
        model = Unet(3, 1).to(device)
    else:
        model = Resnet34_unet(3, 1).to(device)

    train_set = load_dataset(args.data_path, 'train', transform=True)
    criterion = torch.nn.BCEWithLogitsLoss().to(device)
    criterion = DiceBCELoss().to(device)
    optimizer = torch.optim.Adam(model.parameters(), lr = hyperparameters["lr"], weight_decay = hyperparameters["weight_decay"])
    n_epochs = hyperparameters["n_epochs"]
    momentum = 0.99 if args.model == 'unet' else 0.9
    scheduler = torch.optim.lr_scheduler.ExponentialLR(optimizer, momentum)
    writer = SummaryWriter(args.model)

    train_losses = []
    valid_losses = []
    train_dices = []
    valid_dices = []
    train_loader = DataLoader(
        train_set,
        batch_size = args.batch_size,
        shuffle = True,
        drop_last = False,
    )
    gc.collect()
    torch.cuda.empty_cache()
    for epoch in range(hyperparameters["n_epochs"]):
        model.train()

        train_loss = []
        train_dice = []
        for sample in tqdm(train_loader):
            model.train()
            img, mask = sample["image"].to(device), sample["mask"].to(device)
            logits = model(img)

            loss = criterion(logits, mask)

            optimizer.zero_grad()

            loss.backward()

            torch.nn.utils.clip_grad_norm_(model.parameters(), 10)

            optimizer.step()
            model.eval()
            dice = dice_score(logits.to(device), mask).cpu().float().mean()
            train_loss.append(loss.cpu().item())
            train_dice.append(dice.cpu())

        scheduler.step()

        train_loss = sum(train_loss) / len(train_loss)
        train_dice = sum(train_dice) / len(train_dice)

        train_losses.append(train_loss)
        train_dices.append(train_dice)
        writer.add_scalar("Training Accuracy", train_dice, epoch)
        writer.add_scalar("Training Loss", train_loss, epoch)

        print(f"[ Train | {epoch + 1:03d}/{n_epochs:03d} ] loss = {train_loss:.5f}, dice = {train_dice:.5f}")

        valid_set = load_dataset(args.data_path, 'valid')
        valid_loss, valid_dice = evaluate(model, valid_set, device, args, criterion)

        valid_losses.append(valid_loss)
        valid_dices.append(valid_dice)
        writer.add_scalar("Validation Accuracy", valid_dice, epoch)
        writer.add_scalar("Validation loss", valid_loss, epoch)
        print(f"[ Valid | {epoch + 1:03d}/{n_epochs:03d} ] loss = {valid_loss:.5f}, dice = {valid_dice:.5f}")

        if valid_dice > best_dice:
            print(f"Best model found at epoch {epoch }, saving model")
            save_path = f"saved_models/{args.model}"
            os.makedirs(save_path, exist_ok=True)
            torch.save(model.state_dict(), f"{save_path}/{args.model}_best.pth") # only save best to prevent output memory exceed
            best_dice = valid_dice
            stale = 0
        else:
            stale += 1
            if stale > hyperparameters["patience"]:
                print(f"No improvment {hyperparameters['patience']} consecutive epochs, early stopping")
                n_epochs = epoch + 1
                break
        gc.collect()
        torch.cuda.empty_cache()
    writer.close()
    epochs = list(range(n_epochs))
    plot_list = [
        train_losses,
        valid_losses,
        train_dices,
        valid_dices
    ]
    plot(epochs, plot_list)
```

2. evaluate.py:

The evaluate.py script is primarily used for validation and testing to calculate the loss and dice score. Before calculating the loss and dice score, the predicted mask and the ground truth mask are converted to one-hot vectors to facilitate the subsequent computations. Finally, the script outputs the average loss and dice score.

```
def evaluate(model, data, device, args, criterion):
    # implement the evaluation function here
    #data = valid_set
    model.eval()
    eval_loss = []
    eval_dices = []
    eval_loader = DataLoader(
        data,
        batch_size = args.batch_size,
        shuffle = False,
        drop_last = False,
        pin_memory=True
    )

    for sample in tqdm(eval_loader):
        features, mask = sample["image"].to(device), sample["mask"].to(device)

        with torch.no_grad():
            logits = model(features)

            logits = logits.view(logits.size(0), -1)
            mask = mask.view(mask.size(0), -1)

            loss = criterion(logits.to(device), mask.to(device)).to("cpu")
            dice = dice_score(logits, mask).float().mean().to("cpu")

            eval_loss.append(loss.item())
            eval_dices.append(dice)

    eval_loss = sum(eval_loss) / len(eval_loss)
    eval_dice = sum(eval_dices) / len(eval_dices)

    gc.collect()
    torch.cuda.empty_cache()

    return eval_loss, eval_dice
```

3. inference.py

The inference process uses the get_args function to obtain the necessary data. After setting up all conditions, it loads the previously trained best model to perform inference. By using the evaluate function,

it calculates the test loss and test dice score, resulting in the final dice score that we need.

```
def get_args():
    parser = argparse.ArgumentParser(description='Predict masks from input images')
    parser.add_argument('--data_path', type=str, default = os.path.join("dataset/oxford-iiit-pet"), help='path of the input data')
    parser.add_argument('--batch_size', '-b', type=int, default= 6, help='batch size')
    parser.add_argument("--model", "-m", type = str, default = 'resnet34_unet', help = 'UNET or resnet34_unet')

    return parser.parse_args()

if __name__ == '__main__':
    args = get_args()
    test_set = load_dataset(args.data_path, 'test')
    device = "cuda" if torch.cuda.is_available() else "cpu"
    criterion = DiceBCELoss().to(device)
    if args.model == 'UNET':
        model_best = Unet(3, 1).to(device)
    else:
        model_best = Resnet34_unet(3, 1).to(device)

    model_best.load_state_dict(torch.load(f"./saved_models/{args.model}/{args.model}_best.pth"))
    test_loss, test_dice = evaluate(model_best, test_set, device, args, criterion)
    print(f" Test | loss = {test_loss:.5f}, dice = {test_dice:.5f}")
```

B. Details of your model (UNet & ResNet34_UNet)

1. unet.py

First, create a double convolution block as an nn.Module block, which is a basic unit commonly used in UNet. According to the paper, set the Conv2d kernel size to (3, 3) with padding = 1, and use reflection padding to achieve better accuracy. Then, add a batch normalization layer and ReLU activation. The convolution layers are initialized as described in the paper, with the layer weights set to a normal distribution with a standard deviation of $\sqrt{2/(kernel_size^2 * in_channels)}$, and the biases set to 0.

```
class double_conv_block(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(double_conv_block, self).__init__()
        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channel, out_channel, (3, 3), padding=1, padding_mode = "reflect"),
            #add padding for same dim for output
            nn.BatchNorm2d(out_channel),
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(out_channel, out_channel, (3, 3), padding=1, padding_mode= "reflect"),
            nn.BatchNorm2d(out_channel),
            nn.ReLU(inplace=True)
        )

        for layer in self.double_conv:
            if isinstance(layer, nn.Conv2d):
                init.normal_(layer.weight, mean=0.0, std=math.sqrt(2/(9 * layer.in_channels)))
                nn.init.constant_(layer.bias, 0)

    def forward(self, x):
        return self.double_conv(x)
```

Next, separately create `down_conv` and `up_conv` blocks. The former uses max pooling for down sampling, which reduces the receptive field size. The latter uses `ConvTranspose2d` for up sampling, enlarging the receptive field, and applies the same initialization technique as the double conv. Both blocks are followed by a double conv layer.

```
class down_conv(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(down_conv, self).__init__()
        self.maxpool = nn.MaxPool2d(2, 2)
        self.double_conv = double_conv_block(in_channel, out_channel)

    def forward(self, x):
        x = self.maxpool(x)
        x = self.double_conv(x)
        return x

class up_conv(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(up_conv, self).__init__()
        self.up_convolution = nn.ConvTranspose2d(in_channel, in_channel // 2, kernel_size=(2, 2), stride=2)
        init.normal_(self.up_convolution.weight, mean=0.0, std=math.sqrt(2/(9 * in_channel)))
        init.constant_(self.up_convolution.bias, 0)

        self.double_conv = double_conv_block(in_channel, out_channel)

    def forward(self, res_x, x):
        x = self.up_convolution(x)
        x = torch.cat([res_x, x], dim=1)
        x = self.double_conv(x)
        return x
```

During up sampling in `up_conv`, it needs to receive the output preserved from `down_conv`. Therefore, in the forward pass, the output from `down_conv` is concatenated.

Finally, the UNet consists of an input convolution layer, four down-sampling layers, four up-sampling layers, and one output convolution layer. The `input_conv` is a double convolution block that transforms the three input image channels into 64 channels. During the down-sampling process, each layer halves the receptive field and doubles the number of channels.

In the up-sampling process, due to `padding = 1`, the sizes remain symmetric, so no additional cropping is needed for concatenation. Each up-sampling layer doubles the receptive field, concatenates all channels, and then reduces the number of channels by half before outputting to `out_conv`. The `out_conv` is a `Conv2d` layer with a kernel

size of 1, which reduces the final output from 64 channels to a single 256 x 256 output.

```
class Unet(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(Unet, self).__init__()
        self.in_conv = double_conv_block(in_channel, 64)
        self.down1 = down_conv(64, 128)
        self.down2 = down_conv(128, 256)
        self.down3 = down_conv(256, 512)
        self.down4 = down_conv(512, 1024)
        self.up1 = up_conv(1024, 512)
        self.up2 = up_conv(512, 256)
        self.up3 = up_conv(256, 128)
        self.up4 = up_conv(128, 64)
        self.out_conv = nn.Conv2d(64, out_channel, (1, 1))
    def forward(self, x):
        res1 = self.in_conv(x)
        res2 = self.down1(res1)
        res3 = self.down2(res2)
        res4 = self.down3(res3)
        x = self.down4(res4)
        x = self.up1(res4, x)
        x = self.up2(res3, x)
        x = self.up3(res2, x)
        x = self.up4(res1, x)
        logits = self.out_conv(x)
        return logits
```

2. resnet34_unet.py

To begin with, create a basic unit of ResNet, known as resblock, which consists of two convolutional layers. When the stride is not equal to 1 or when the number of input channels does not match the number of output channels, down-sampling is necessary. As described in the ResNet paper, use a convolution with a kernel size of (1, 1) to perform down-sampling on the identity shortcut. In the forward pass, the identity is skipped over the double convolution and added to the output to form the residual connection.

```

class res_block(nn.Module):
    def __init__(self, in_channel, out_channel, stride = 1):
        super(res_block, self).__init__()
        self.stride = stride

        self.double_conv = nn.Sequential(
            nn.Conv2d(in_channel, out_channel, (3, 3), stride = self.stride, padding=1),
            nn.BatchNorm2d(out_channel),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_channel, out_channel, (3, 3), stride = 1, padding=1),
            nn.BatchNorm2d(out_channel)
        )

        self.downsample = None
        if self.stride != 1 or in_channel != out_channel:
            self.downsample = nn.Sequential(
                nn.Conv2d(in_channel, out_channel, (1, 1), stride = self.stride),
                nn.BatchNorm2d(out_channel)
            )

        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        identity = x
        x = self.double_conv(x)
        if self.downsample is not None:
            identity = self.downsample(identity)

        x += identity
        x = self.relu(x)
        return x

```

Next, implement the concatenation and up-sampling process. Similar to UNet, during up-sampling in ResNet34_UNet, concatenate the output from the down-sampling stage with the corresponding up-sampling channels. The concatenation allows the up-sampling stage to utilize features from the down-sampling stage. The initialization method for the convolution layers should be consistent with the one used in UNet.

```

class concat_upsample(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(concat_upsample, self).__init__()
        self.up_convolution = nn.ConvTranspose2d(in_channel, out_channel, kernel_size=(2, 2), stride=2)
        init.normal_(self.up_convolution.weight, mean=0.0, std= math.sqrt(2/(9 * in_channel)))
        init.constant_(self.up_convolution.bias, 0)

        self.conv = nn.Sequential(
            nn.Conv2d(out_channel, out_channel, (3, 3), padding=1),
            nn.BatchNorm2d(out_channel),
            nn.ReLU(inplace=True)
        )

    def forward(self, res_x, x):
        x = torch.cat([res_x, x], dim=1)
        x = self.up_convolution(x)
        x = self.conv(x)
        return x

```

For the Resnet34_uNet architecture includes:

1. Input Convolution Layer:

Utilizes a (7, 7) kernel with a stride of 3 and max pooling, resulting in 64 channels and a 64 x 64 feature map.

2. Residual Layers:

Constructed with the make_layer function, featuring five residual blocks with varying dimensions and feature maps. Identity down-sampling is applied only in the first layer of each main block (except the first main block), as described in the ResNet paper.

3. Concat and Up-Sample Layers:

Four layers for concatenation and up-sampling, similar to UNet. The feature map sizes are designed to match on both sides, so no cropping or padding is needed. Each layer, excluding the skip connections, contains 32 channels.

4. Output Convolution:

Uses residual blocks to produce a final output with a single channel and a 256 x 256 feature map.


```

class Resnet34_unet(nn.Module):
    def __init__(self, in_channel, out_channel):
        super(Resnet34_unet, self).__init__()
        self.input_conv = nn.Sequential(
            #B * 3 * 256 * 256
            nn.Conv2d(in_channel, 64, (7, 7), stride=2, padding=3),
            #B * 64 * 128 * 128
            nn.BatchNorm2d(64),
            nn.ReLU(inplace=True),
            nn.MaxPool2d((3, 3), stride=2, padding=1)
            #B * 64 * 64 * 64
        )
        self.res_1 = self.make_layer(64, 64, 3, stride = 1)
        # first layer don't need downsample
        #B * 64 * 64 * 64
        self.res_2 = self.make_layer(64, 128, 4, stride = 2)
        #B * 128 * 32 * 32
        self.res_3 = self.make_layer(128, 256, 6, stride = 2)
        #B * 256 * 16 * 16
        self.res_4 = self.make_layer(256, 512, 3, stride = 2)
        #B * 512 * 8 * 8
        self.res_5 = self.make_layer(512, 256, 1, stride = 1)
        # B * 256 * 8 * 8
        #+B * 512 * 8 * 8
        #=B * 768 * 8 * 8
        self.up1 = concat_upsample(256 + 512, 32)
        #B * 32 * 16 * 16
        self.up2 = concat_upsample(32 + 256, 32)
        #B * 32 * 32 * 32
        self.up3 = concat_upsample(32 + 128, 32)
        #B * 32 * 64 * 64
        self.up4 = concat_upsample(32 + 64, 32)
        #B * 32 * 128 * 128
        self.out_conv = nn.Sequential(
            nn.ConvTranspose2d(32, 32, kernel_size=(2, 2), stride=2),
            nn.Conv2d(32, 32, (3, 3), padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(inplace=True),
            res_block(32, out_channel, stride = 1)
        )

    def make_layer(self, in_channel, out_channel, block_num, stride = 2):
        res_layers = []
        res_layers.append(res_block(in_channel, out_channel, stride = stride))
        #for downsample
        for i in range(block_num - 1):
            res_layers.append(res_block(out_channel, out_channel, stride = 1))
        #others
        return nn.Sequential(*res_layers)

    def forward(self, x):
        x = self.input_conv(x)
        res1 = self.res_1(x)
        res2 = self.res_2(res1)
        res3 = self.res_3(res2)
        res4 = self.res_4(res3)
        out = self.res_5(res4)
        out = self.up1(res4, out)
        out = self.up2(res3, out)
        out = self.up3(res2, out)
        out = self.up4(res1, out)
        out = self.out_conv(out)
        return out

```

C. Anything more you want to mention:

DiceBCELoss:

During training, you use DiceBCELoss, which is a combination of Binary Cross Entropy Loss (BCE) and Dice Loss. Since the loss function receives logits, they are first converted into one-hot vectors. After applying the sigmoid function to obtain probabilities between 0 and 1, the Dice score is calculated. The output is 1-Dice score. BCE is handled using PyTorch's built-in `binary_cross_entropy_with_logits`.

3. Data preprocessing

A. How you preprocessed your data

I handle image and mask augmentation by applying rotations, horizontal flips, and vertical flips. For rotations, I use the `rotate` function from `scipy.ndimage` to simultaneously rotate images, masks, and trimaps. For horizontal and vertical flips, I use `np.flipud` and `np.fliplr`, respectively.

Original: Rotate(d = 7): vertical flip: horizon flip:



B. What makes your method unique

I handle data augmentation in the `get_item` function of your dataset class. This means that each time the `DataLoader` retrieves a sample, it randomly determines the rotation angle and whether to apply horizontal and vertical flips. The rotation angle is randomly chosen from integers between 0 and 15, and horizontal and vertical flip probabilities are set as random floats between 0 and 1. This ensures that each image is uniquely augmented each time it is retrieved.

```

if self.transform:
    degree = np.random.randint(0, 15)
    sample['image'] = rotate(sample["image"], degree, reshape=False)
    sample['mask'] = rotate(sample["mask"], degree, reshape=False)
    sample['trimap'] = rotate(sample["trimap"], degree, reshape=False)

    verticle_flip_p = np.random.uniform(0, 1)
    if verticle_flip_p >= 0.5:
        sample['image'] = np.flipud(sample['image'])
        sample['mask'] = np.flipud(sample['mask'])
        sample['trimap'] = np.flipud(sample['trimap'])

    horizon_flip_p = np.random.uniform(0, 1)
    if horizon_flip_p >= 0.5:
        sample['image'] = np.fliplr(sample['image'])
        sample['mask'] = np.fliplr(sample['mask'])
        sample['trimap'] = np.fliplr(sample['trimap'])

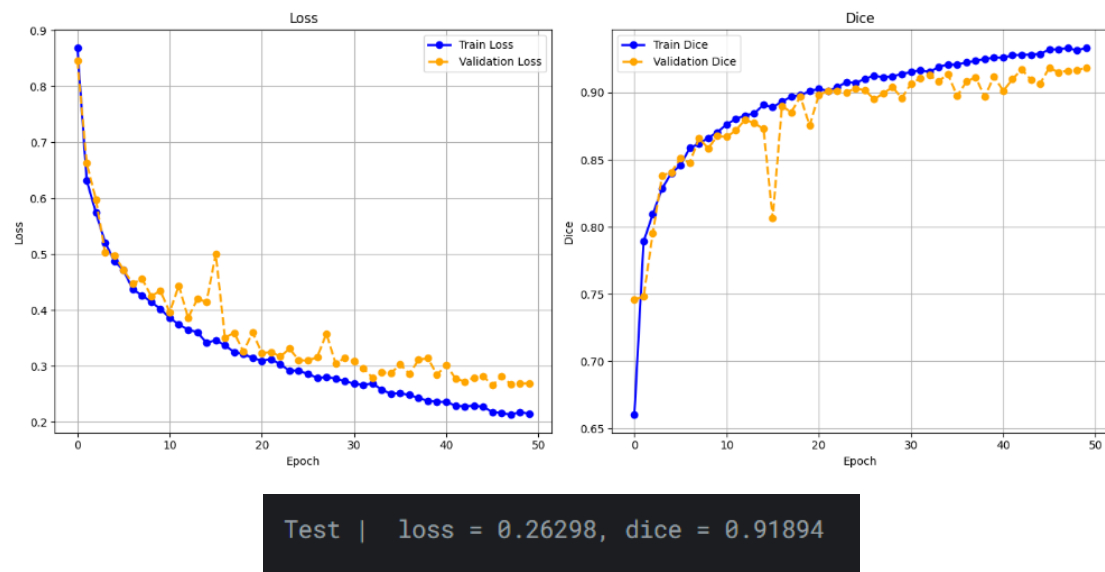
```

4. Analyze on the experiment results

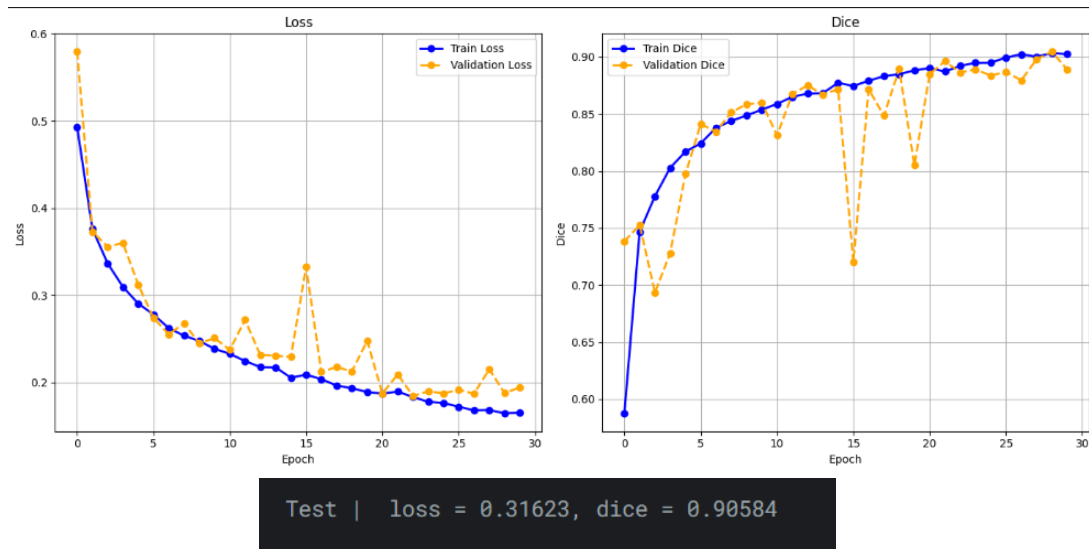
A. What did you explore during the training process

In this lab training, I observed that different loss functions have varying impacts on performance. Both UNet and ResNet34_UNet performed better with DiceBCELoss compared to using only BCELoss, likely due to the improved Dice score performance from adding Dice Loss. Cross Entropy Loss was the least effective, but only slightly worse than BCE. This might be because BCE is more appropriate for binary classification tasks.

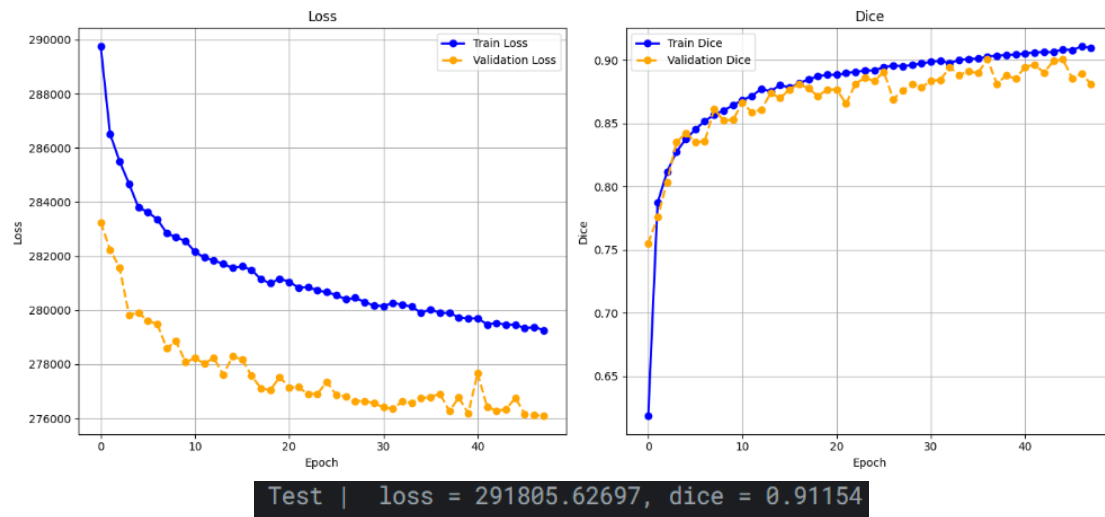
Unet with DiceBCELoss:



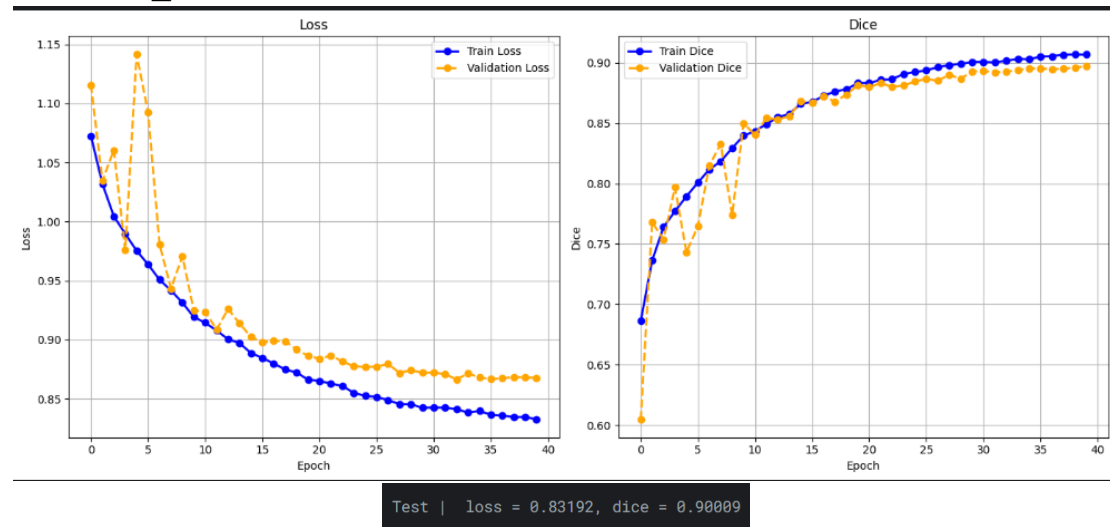
Unet with BCELoss:



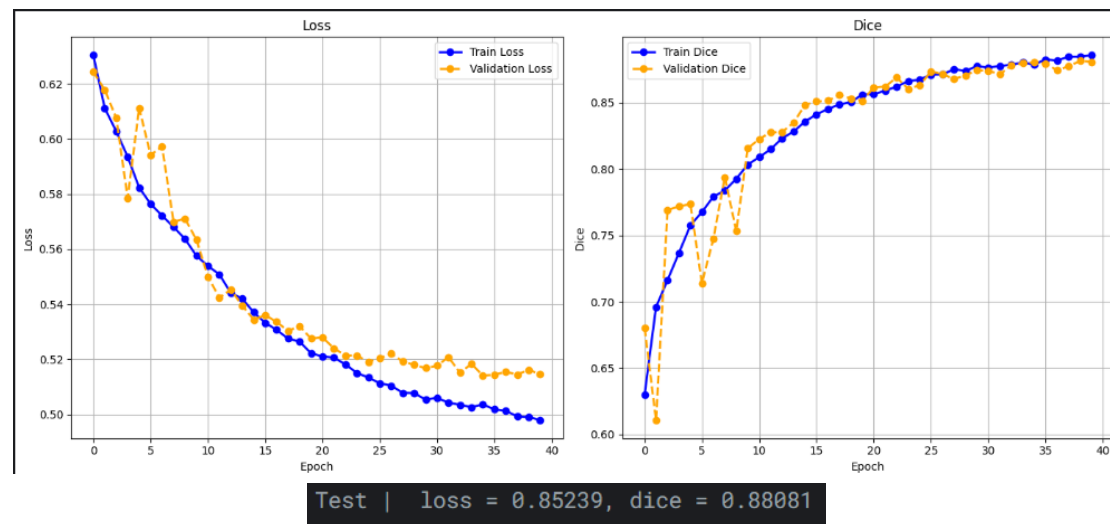
Unet with Cross entropy Loss:



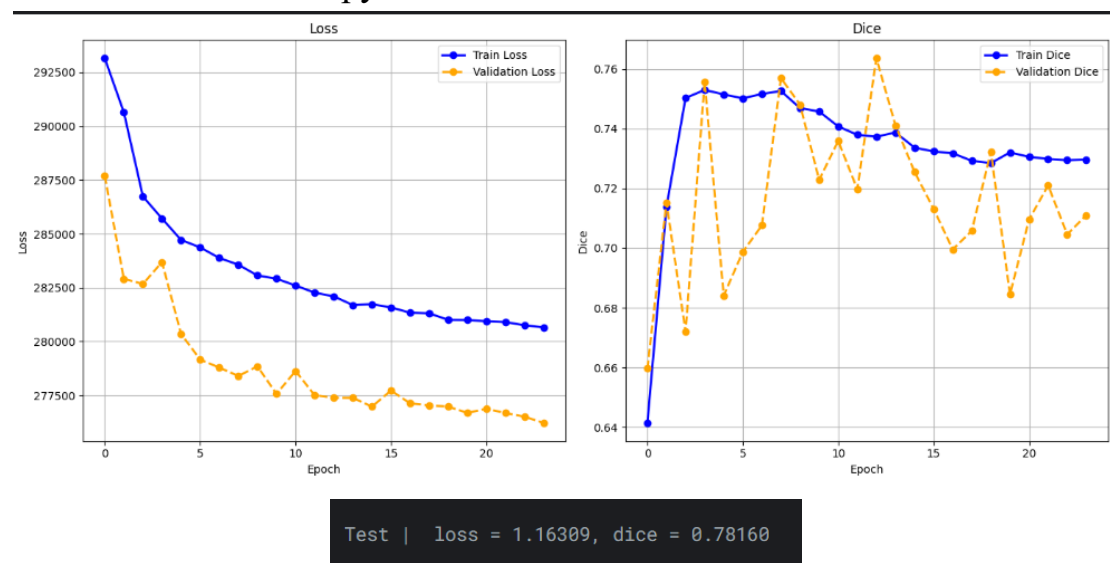
Resnet34_unet with DiceBCELoss:



Resnet34_unet with BCELoss:



Unet with Cross entropy Loss:



1. Batch_size:

Although the ResNet34_UNet paper recommends a batch size of 6, I found that using a batch size of 16 yielded better dice scores on average.

Batch_size = 6

```
Test | loss = 0.83192, dice = 0.90009
```

Batch_size = 16

```
Test | loss = 0.82635, dice = 0.90363
```

2. Training time

UNet typically requires around 50 epochs to achieve a dice score above 0.9, whereas ResNet34_UNet can reach similar performance in about 40 epochs. Additionally, ResNet34_UNet has fewer parameters than UNet, leading to faster computation.

B. Found any characteristics of the data

I am working on identifying images with lower dice scores and analyzing them to understand the potential reasons for their lower performance.

1. Color

When animals have colors similar to the background, it can make it challenging for the model to distinguish between them, leading to lower dice scores. This situation often results in poor segmentation performance because the model may have difficulty differentiating between the foreground (animal) and the background due to their similar color features.

Dice score = 0.4366

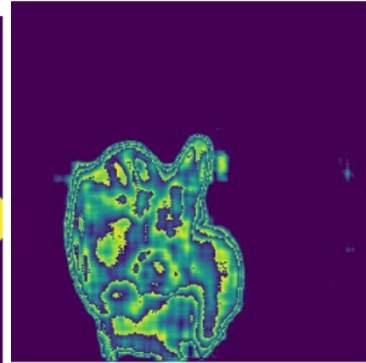
Input image:



ground truth mask:



predict mask:

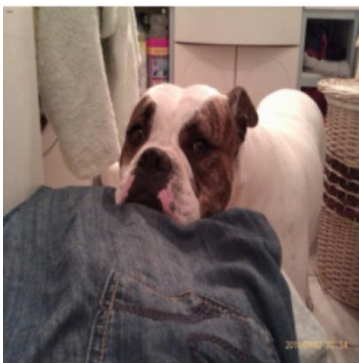


2. Irregular shapes

Irregular shapes and connected objects can make it difficult to determine boundaries, leading to lower dice scores.

Dice score = 0.4123

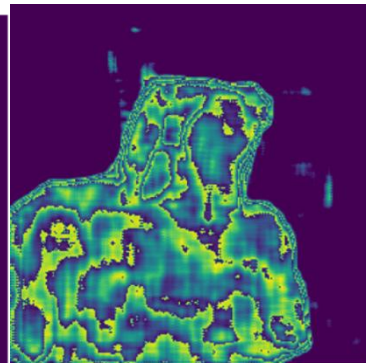
Input image:



ground truth mask:



predict mask:

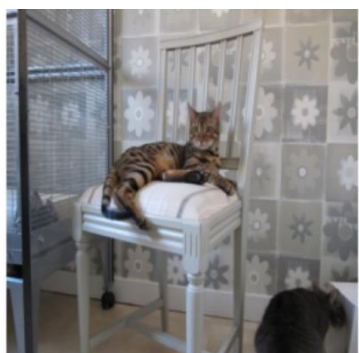


3. Blur

Blurry photos can lead to lower dice scores because the lack of clear edges and details makes it difficult for the model to accurately segment the image

Dice score = 0.4123

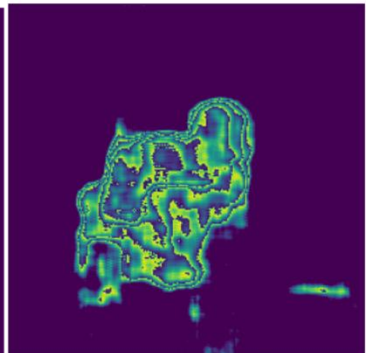
Input image:



ground truth mask:



predict mask:



5. Execution command

You can simply set `args.model` to `unet` or `resnet34_unet`, or manually input `-model r` in the command line. The other hyperparameters will automatically adjust based on the selected model. Here is the hyperparameter set that I use for each model in training.

Train:

Unet:

```
def get_args():
    parser = argparse.ArgumentParser(description='Train the UNet on images and target masks')
    parser.add_argument('--data_path', type=str, default = os.path.join("dataset/oxford-iiit-pet"), help='path of the input data')
    parser.add_argument('--batch_size', '-b', type=int, default= 6, help='batch size')
    parser.add_argument("--model", "-m", type = str, default = 'unet', help = 'unet or resnet34_unet')

    return parser.parse_args()
```

batch size = 16

learning rate = 0.0003

total epochs = 50

patience(for early stopping):10

weight_decay = 0.0001

```
def get_hyperparameters(model):
    assert model in {'unet', 'resnet34_unet'}
    hyperparameters = {}
    if model == 'unet':
        hyperparameters['lr'] = 0.0003
        hyperparameters['n_epochs'] = 50
        hyperparameters['patience'] = 10
        hyperparameters['weight_decay'] = 0.0001
    else:
```

Resnet34_unet:

```
def get_args():
    parser = argparse.ArgumentParser(description='Train the UNet on images and target masks')
    parser.add_argument('--data_path', type=str, default = os.path.join("dataset/oxford-iiit-pet"), help='path of the input data')
    parser.add_argument('--batch_size', '-b', type=int, default= 16, help='batch size')
    parser.add_argument("--model", "-m", type = str, default = 'resnet34_unet', help = 'unet or resnet34_unet')

    return parser.parse_args()
```

batch size = 16

learning rate = 0.001

total epochs = 40

patience(for early stopping):10

weight decay = 0.0001

```
else:
    hyperparameters['lr'] = 0.001
    hyperparameters['n_epochs'] = 40
    hyperparameters['patience'] = 10
    hyperparameters['weight_decay'] = 0.0001
```

Inference:

Just like in training, you need to modify `args.model` to the model type you want to use. Make sure you have the previously trained model saved in `save_models` to proceed with the process.

```
def get_args():
    parser = argparse.ArgumentParser(description='Predict masks from input images')
    parser.add_argument('--data_path', type=str, default = os.path.join("dataset/oxford-iiit-pet"), help='path of the input data')
    parser.add_argument('--batch_size', '-b', type=int, default= 16, help='batch size')
    parser.add_argument("--model", "-m", type = str, default = 'resnet34_unet', help = 'unet or resnet34_unet')

    return parser.parse_args()
```

batch size = 16

model = unet or resnet34_unet

6. Discussion

A. What architecture may bring better results

Based on the results of this lab, I believe that on average, UNet has a higher accuracy (dice score). However, its drawback is that it is larger and requires more training time compared to ResNet34_UNet. If aiming for better performance, there are two directions that could potentially increase the dice score in future attempts:

Elastic Deformation for Data Augmentation: As suggested in the UNet paper, applying elastic deformation can enhance data augmentation. However, this will likely increase the required training time.

Weighting Loss Technique: Another approach from the UNet paper involves using a trimap to generate a weight map. This weight map assigns higher weights to boundary regions, thereby increasing the loss at the boundaries. This encourages the model to learn boundary information more effectively.

B. What are the potential research topics in this task

In this lab, we observed that combining Unet with other network architectures could be a promising research topic. Binary semantic segmentation is commonly used in medical imaging, where labeled data is scarce and often requires significant human effort to obtain. Therefore, exploring semi-supervised or unsupervised learning approaches is worthwhile.

Moreover, in medical imaging, real-time segmentation is crucial. It's important to investigate ways to reduce model complexity and increase computational speed while maintaining accuracy. Balancing these aspects is another valuable area for future research.